Problem Statement

Porter is India's largest intra-city logistics marketplace, handling delivery of goods, especially food, from restaurants to customers. The company has a large pool of delivery partners and wants to estimate delivery times for orders using historical data. This is important because:

Customers want accurate delivery time estimates.

Efficient allocation of delivery partners can reduce delays and optimize operations.

Insights can improve driver earnings and customer satisfaction.

Goal:

Building a regression model that predict the estimated delivery time for food orders in Porter's intra-city logistics network. Using historical delivery data, the model will estimate the time from order placement to actual delivery based on features like order details (e.g., items, prices), restaurant category, market, order protocol, partner availability, and estimated driving duration. Accurate predictions help set customer expectations, optimize partner allocation, and improve service reliability.

This approach can be used in e-commerce/logistics platforms (e.g., Uber Eats, DoorDash) for real-time ETA forecasting. Modifications could extend to multi-modal transport (e.g., incorporating traffic APIs for dynamic ETAs) or predictive maintenance for delivery vehicles.

Dataset Overview

Each row represents one delivery. Key columns:

Column	Description
market_id	ID of the market where the restaurant is located
created_at	Timestamp when the order was placed
actual_delivery_time	Timestamp when the order was delivered
store_primary_category	Restaurant category (e.g., fast food, cafe)
order_protocol	Code indicating how the order was placed (app, call, third-party)
total_items_subtotal	Final order price
num_distinct_items	Number of distinct items in the order
min_item_price	Price of cheapest item
max_item_price	Price of most expensive item
total_onshift_partners	Delivery partners available at order time
total_busy_partners	Delivery partners currently busy
total_outstanding_orders	Orders pending at that moment
estimated_store_to_consumer_driving_duration	Predicted travel time from restaurant to customer

In [55]:
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.neighbors import LocalOutlierFactor
from sklearn.impute import KNNImputer

from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import mean_absolute_error, r2_score

from statsmodels.stats.outliers_influence import variance_inflation_factor

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation, LeakyReLU, Dropout, Bat
from tensorflow.keras.optimizers import Adam, RMSprop
from tensorflow.keras.utils import plot_model

from sklearn.metrics import mean_squared_error

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_percentage_error
```

In [6]:
```
# Data Loading

df=pd.read_csv('data_2.csv')
df.head()
```

Out[6]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1	06-02-2015 22:24	06-02-2015 23:11	4	
1	2	10-02-2015 21:49	10-02-2015 22:33	46	
2	2	16-02-2015 00:11	16-02-2015 01:06	36	
3	1	12-02-2015 03:36	12-02-2015 04:35	38	
4	1	27-01-2015 02:12	27-01-2015 02:58	38	

In [7]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  int64
 1   created_at                                    175777 non-null  object
 2   actual_delivery_time                          175777 non-null  object
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  int64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  int64
 11  total_busy_dashers                            175777 non-null  int64
 12  total_outstanding_orders                      175777 non-null  int64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  int64
dtypes: int64(12), object(2)
memory usage: 18.8+ MB
```

The dataset contains 175,777 rows and 14 columns, meaning there are 175,777 delivery records.

There are 12 numerical columns (int64) and 2 object columns.

The columns created_at and actual_delivery_time are stored as object data type, which indicates they are timestamps stored as strings.

These should be converted to datetime format for further analysis and feature engineering.

All columns have 175,777 non-null values, which indicates that there are no missing values in the dataset.

Several features relate to:

Order characteristics (total_items, subtotal, num_distinct_items)

Pricing information (min_item_price, max_item_price)

Delivery partner availability (total_onshift_dashers, total_busy_dashers)

System workload (total_outstanding_orders)

Travel time estimate (estimated_store_to_consumer_driving_duration)

The memory usage of the dataset is 18.8 MB, which is manageable for analysis and model training.

In [8]:
```
df.isna().sum()
```

Out[8]:

	0
market_id	0
created_at	0
actual_delivery_time	0
store_primary_category	0
order_protocol	0
total_items	0
subtotal	0
num_distinct_items	0
min_item_price	0
max_item_price	0
total_onshift_dashers	0
total_busy_dashers	0
total_outstanding_orders	0
estimated_store_to_consumer_driving_duration	0

dtype: int64

The missing value check shows that all columns contain 0 null values.

This means the dataset is complete and does not contain any missing data.

Since there are no null values, no imputation techniques such as mean/median filling or KNN imputation are required at this stage.

All 175,777 records can be used for analysis and model training without dropping any rows due to missing values.

In [9]:
```
df.nunique()
```

Out[9]:

	0
market_id	6
created_at	23362
actual_delivery_time	23420
store_primary_category	73
order_protocol	7
total_items	54
subtotal	8182
num_distinct_items	20
min_item_price	2251
max_item_price	2585
total_onshift_dashers	172
total_busy_dashers	158
total_outstanding_orders	281
estimated_store_to_consumer_driving_duration	1318

dtype: int64

The df.nunique() function was used to determine the number of unique values in each column.

The market_id column has only 6 unique values, indicating the dataset contains orders from six different markets.

The timestamp columns (created_at and actual_delivery_time) have a very high number of unique values, which is expected since each order occurs at a different time.

The store_primary_category column has 73 unique categories, representing

different types of restaurants.

The order_protocol column has 7 unique values, indicating the order can be placed through several different ordering methods.

Order-related features such as total_items, subtotal, and num_distinct_items have multiple unique values representing different order sizes and costs.

Delivery partner availability features like total_onshift_dashers, total_busy_dashers, and total_outstanding_orders also show a wide range of values, indicating varying system load conditions.

The estimated_store_to_consumer_driving_duration column has 1318 unique values, representing different estimated travel times between restaurants and customers.

The dataset contains a mix of categorical and numerical features. Columns like market_id, store_primary_category, and order_protocol have limited unique values and represent categorical variables, while order and delivery-related features have a larger range of unique values representing different operational conditions.

In [10]:
```
df.duplicated().value_counts()
```

Out[10]:

	count
False	175776
True	1

dtype: int64

Interpretation

175,776 rows are unique

1 row is duplicated

In [13]:
```
# Removing the duplicate record
df = df.drop_duplicates()
```

In [18]:
```
# After removing duplicates
df.duplicated().sum()
```

Out[18]:
```
np.int64(0)
```

Duplicate records were checked using df.duplicated(). It was found that the dataset contained 1 duplicate row. This duplicate entry was removed using

drop_duplicates() to maintain data integrity and ensure that each delivery record appears only once in the dataset.

In [19]:
```
df.head(5)
```

Out[19]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1	06-02-2015 22:24	06-02-2015 23:11	4	
1	2	10-02-2015 21:49	10-02-2015 22:33	46	
2	2	16-02-2015 00:11	16-02-2015 01:06	36	
3	1	12-02-2015 03:36	12-02-2015 04:35	38	
4	1	27-01-2015 02:12	27-01-2015 02:58	38	

In [21]:
```
# Creating Target Column

df['created_at'] = pd.to_datetime(df['created_at'], format='%d-%m-%Y %H:%M')
df['actual_delivery_time'] = pd.to_datetime(df['actual_delivery_time'], format
```

In [22]:
```
# Delivery Time Calculation

df['Time_taken_for_delivery'] = (df['actual_delivery_time'] - df['created_at']
```

In [23]:
```
df['hour'] = df['created_at'].dt.hour
df['day'] = df['created_at'].dt.day
df['month'] = df['created_at'].dt.month
```

In [24]:
```
# Dropping the Column that are no longer required

df.drop(['created_at','actual_delivery_time'], axis=1, inplace=True)
```

In [25]:
```
# Encoding store_primary_category

df['store_primary_category'] = df['store_primary_category'].astype('category')
```

The timestamp columns were converted to datetime format and used to create a new target variable representing delivery time in minutes. Additional temporal features such as order hour and day of the week were extracted to capture time-based delivery patterns. After feature extraction, the original timestamp columns were removed. The store_primary_category column was encoded into numerical category codes to make it suitable for machine learning models.

In [29]:
```
df.columns
```

Out[29]:
```
Index(['market_id', 'store_primary_category', 'order_protocol', 'total_item
s',
       'subtotal', 'num_distinct_items', 'min_item_price', 'max_item_price',
       'total_onshift_dashers', 'total_busy_dashers',
       'total_outstanding_orders',
       'estimated_store_to_consumer_driving_duration',
       'Time_taken_for_delivery', 'hour', 'day', 'month'],
      dtype='object')
```

Correlation Analysis

Why we do this:

To understand relationships between numeric features in the dataset.

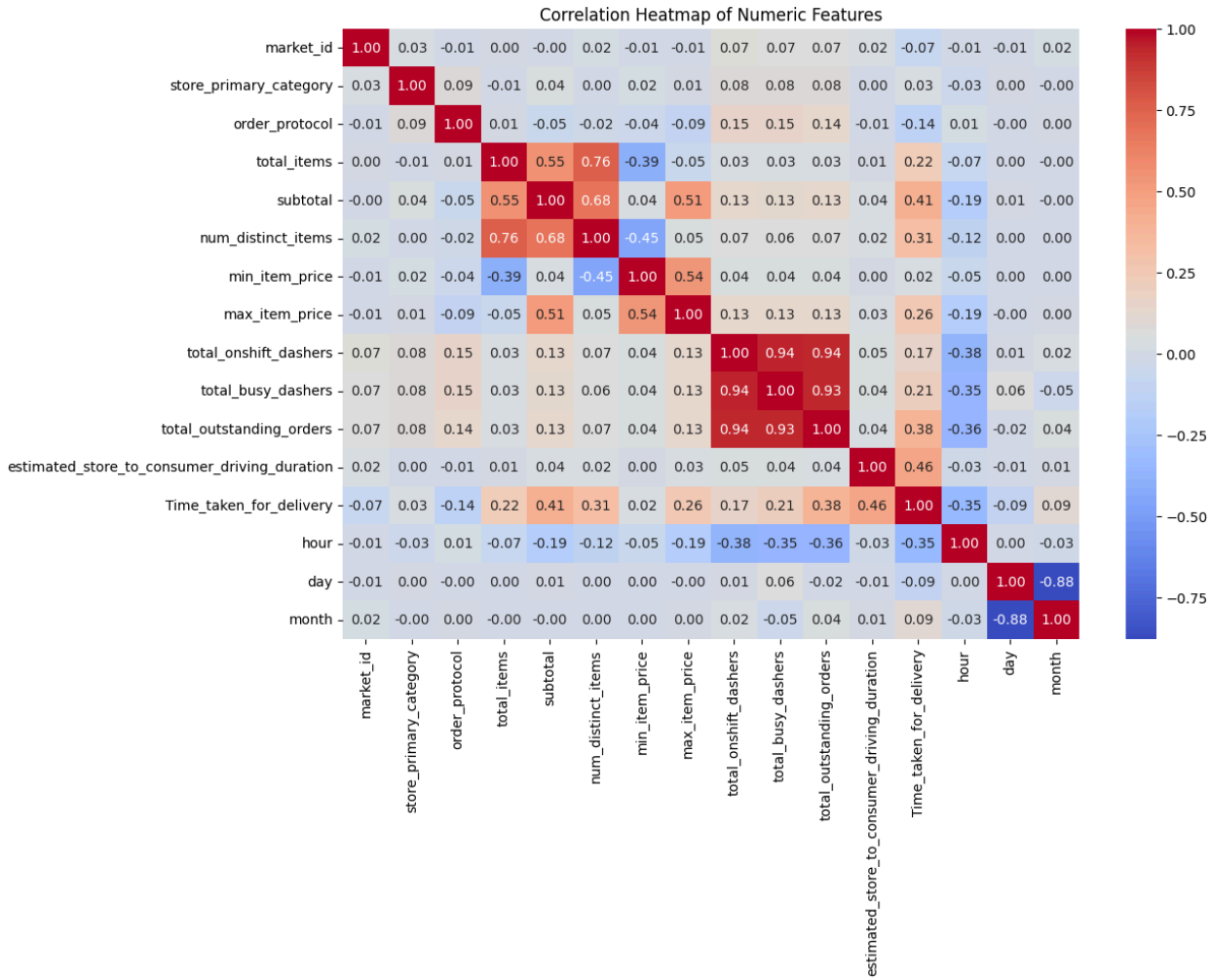
To identify which features are highly correlated with the target (Time_taken_for_delivery), which are likely the most important for prediction.

To detect multicollinearity (features highly correlated with each other) that may affect some models.

To get insights for feature selection or feature engineering before modeling.

In [30]:
```
# Plot histogram of delivery time

plt.figure(figsize=(10,6))
sns.histplot(df['Time_taken_for_delivery'], bins=50, kde=True)
plt.title('Distribution of Delivery Time (minutes)')
plt.xlabel('Delivery Time (minutes)')
plt.ylabel('Number of Orders')
plt.show()
```

In [31]:
```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Correlation Heatmap of Numeric Features")
plt.show()
```



In [32]:
```
df.corr()['Time_taken_for_delivery'].sort_values(ascending=False)
```

Out[32]:

	Time_taken_for_delivery
Time_taken_for_delivery	1.000000
estimated_store_to_consumer_driving_duration	0.460178
subtotal	0.413293
total_outstanding_orders	0.384981
num_distinct_items	0.312078
max_item_price	0.255194
total_items	0.224874
total_busy_dashers	0.206032
total_onshift_dashers	0.171004
month	0.091507
store_primary_category	0.026982
min_item_price	0.022762
market_id	-0.073010
day	-0.085648
order_protocol	-0.136758
hour	-0.346361

dtype: float64

We computed the correlation of numeric features with the target Time_taken_for_delivery to identify which factors most influence delivery time. Features like estimated_store_to_consumer_driving_duration and subtotal are positively correlated, meaning higher values tend to increase delivery time. Features like hour are negatively correlated, indicating faster deliveries at certain times of day.

Correlation of Features with Delivery Time

The correlation of each numeric feature with the target variable Time_taken_for_delivery provides insights into which factors most strongly influence delivery time:

Strong Positive Correlations

estimated_store_to_consumer_driving_duration (0.46) → Longer driving distances naturally lead to longer delivery times.

subtotal (0.41) → Orders with higher subtotal (likely larger or more complex orders) tend to take more time.

total_outstanding_orders (0.38) → When there are more pending orders, delivery time increases due to higher workload.

num_distinct_items (0.31) → Orders with more unique items take longer to prepare and deliver.

Moderate Positive Correlations

max_item_price (0.25) and total_items (0.22) → Expensive or larger orders require slightly more time.

total_busy_dashers (0.21) and total_onshift_dashers (0.17) → More busy/on-shift delivery partners slightly affect delivery efficiency.

Weak Positive Correlations

month (0.09), store_primary_category (0.03), min_item_price (0.02) → Minimal effect on delivery time.

Negative Correlations

hour (-0.35) → Certain hours (likely non-rush hours) have faster deliveries.

order_protocol (-0.14), day (-0.08), market_id (-0.07) → These features show a small inverse relationship; for example, some order methods or markets may handle deliveries more efficiently.

Key Insights

Distance and workload matter most: estimated_store_to_consumer_driving_duration and total_outstanding_orders are the top contributors.

Order complexity also affects time: subtotal and num_distinct_items indicate larger/more complex orders take longer.

Time-of-day effect: Negative correlation with hour suggests that rush hours significantly impact delivery times.

Other features like store category or market_id have minimal direct impact but may still provide minor contextual information.

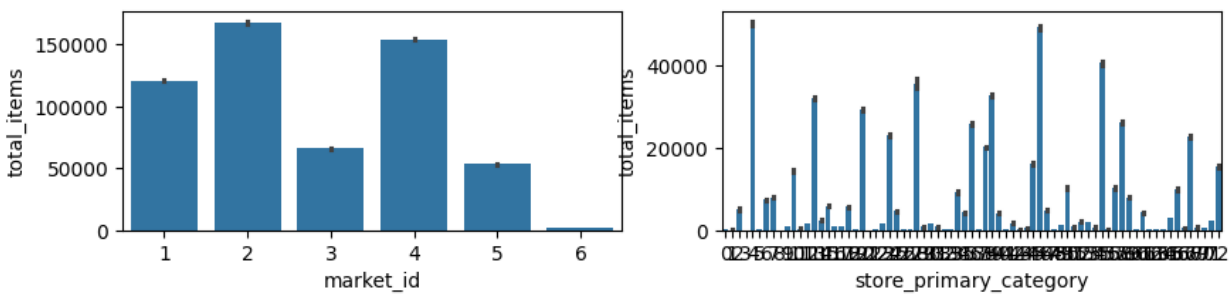
Feature Analysis

What are we going to do:

Plot Type	Category	Purpose
Total items by market/store/order_protocol	Categorical Feature Analysis	Identify workload hotspots and popular markets/categories
Countplots of day/hour/Month	Temporal Analysis	Detect peak days/hours affecting delivery time
Scatterplot: hour vs delivery time	Feature-Target Relationship	Observe effect of time-of-day on delivery time

In [33]:
```
# Order Volume by Market and Restaurant Category

plt.figure(figsize=(10,2))
plt.subplot(1,2,1)
sns.barplot(y='total_items',x='market_id',data=df,estimator='sum')
plt.subplot(1,2,2)
sns.barplot(y='total_items',x='store_primary_category',data=df,estimator='sum'
```

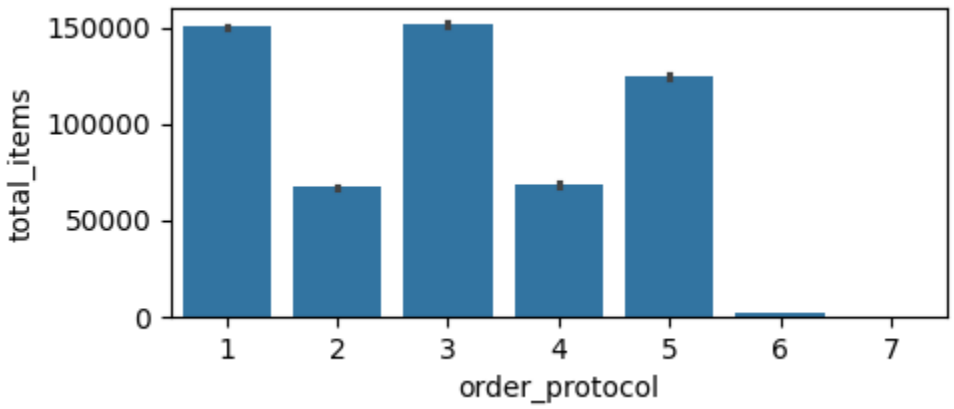
Out[33]: <Axes: xlabel='store_primary_category', ylabel='total_items'>



Aspect	Key Observation	Business / Model Implication
Market concentration	3 markets (1,2,4) dominate ~70–80% of volume	Focus operations, partner onboarding, marketing heavily on these 3 markets first
Market 6 near-zero	Practically inactive	Investigate: new market? Poor restaurant coverage? Data issue?
Category concentration	Very few categories drive most orders	Prioritize restaurant onboarding & promotions in top 5–7 categories
Long tail categories	Many categories with tiny volume	May not be worth heavy investment; consider de-prioritizing or niche marketing only
Data quality note	Market 6 + many near-zero categories suggest possible sparse data / cold-start issues	Important for modeling: consider grouping rare categories or using target encoding

In [34]:
```
# Total Items Ordered by Order Protocol

plt.figure(figsize=(5,2))
sns.barplot(y='total_items',x='order_protocol',data=df,estimator='sum')
```

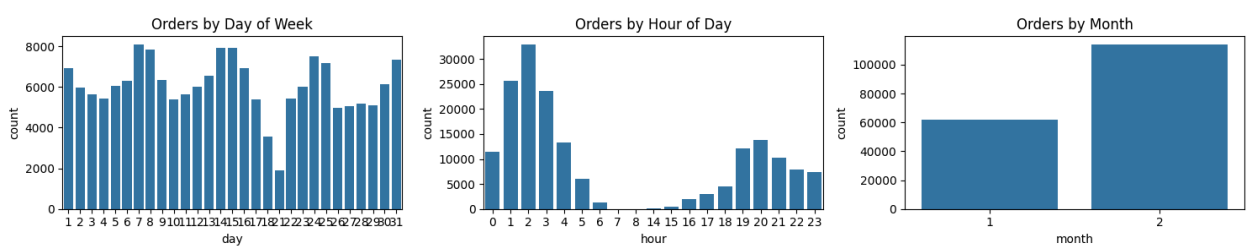
Out[34]: <Axes: xlabel='order_protocol', ylabel='total_items'>



Protocol ID	Likely Meaning (most common mapping)	Volume Level	Business Insight
1	Direct app/website order (Porter customer app)	Highest	Core business channel - most reliable & highest-margin source
3	Third-party aggregator / partner platform	Very high	Major growth driver - important partnerships (possibly Swiggy, Zomato, or local players)
5	Pre-booked / scheduled orders	High	Strong recurring / planned demand - good for predictable partner scheduling
2	Call to restaurant / phone-in orders	Medium	Traditional / legacy channel - still relevant but declining importance
4	Corporate / bulk / institutional orders	Medium	Niche but potentially high-value (offices, events) - worth nurturing
6 & 7	Rare / experimental / deprecated channels (e.g., WhatsApp bot, new API, test mode, walk-in)	Near-zero	Practically inactive - can likely be dropped or investigated for bugs/data issues

In [37]:

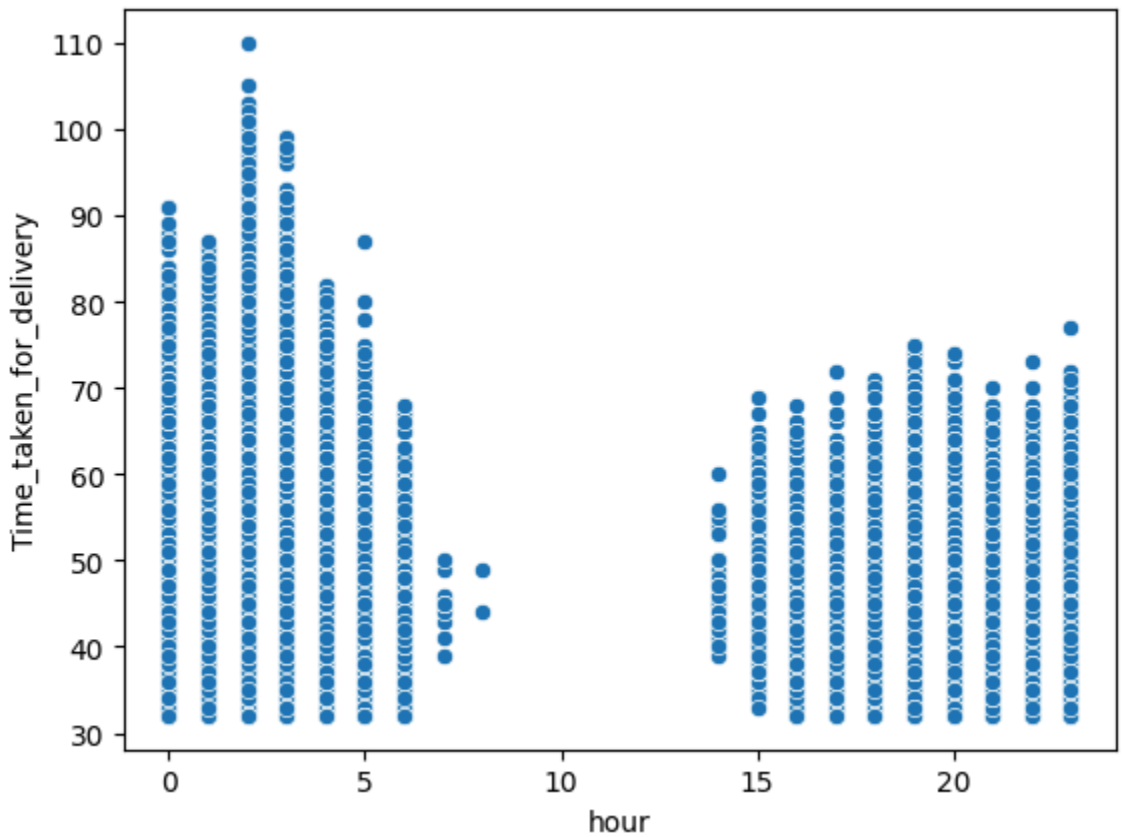
```
# Order Distribution by Day, Hour and Month

plt.figure(figsize=(15,3))

# Orders by day of week
plt.subplot(1,3,1)
sns.countplot(x='day', data=df)
plt.title("Orders by Day of Week")

# Orders by hour of day
plt.subplot(1,3,2)
sns.countplot(x='hour', data=df)
plt.title("Orders by Hour of Day")

# Orders by month
plt.subplot(1,3,3)
sns.countplot(x='month', data=df)
plt.title("Orders by Month")

plt.tight_layout()
plt.show()
```



Aspect	Pattern Observed	Implication / Takeaway
Day of week	Quite balanced across week	No strong weekend bias â†' consistent daily partner requirement
Hour of day	Very strong lunch peak + moderate dinner	Fleet / partner scheduling should heavily prioritize 12â€"15 and 18â€"22 slots
Late-night / early morning	Almost no demand	Very low need for 24/7 operations in this dataset/market
Month-over-month	~70â€"80% jump from month 1 â†' 2	Either hyper-growth phase or partial month 1 data
General demand shape	Lunch-dominant food delivery market	Different from dinner-heavy markets (e.g., many US/metro cities)

In [38]:
```
sns.scatterplot(x='hour',y='Time_taken_for_delivery',data=df)
```

Out[38]:
```
<Axes: xlabel='hour', ylabel='Time_taken_for_delivery'>
```



Night/early morning (0–6): Slowest deliveries (avg 80–110 min), highest variability → low partner availability & tougher conditions.

Daytime/evening (15–22): Fastest & most consistent (avg 55–75 min) → peak efficiency during lunch & dinner rushes.

Mid-morning to afternoon (9–14): Almost no orders → very low demand period.

Main takeaway: Delivery speed is strongly time-dependent → prioritize partner incentives at night, promise realistic ETAs late-night, and exploit fast daytime performance for better customer trust.

In one line:

Outliers treatment

1⃢ Why do we need to check for outliers in our data?

Outliers are extreme values that deviate significantly from the majority of the data.

They can skew statistical measures like mean and variance, affecting model performance.

In regression tasks like predicting delivery time:

Extreme delivery times, huge orders, or unusually high item prices can bias the model, leading to poor predictions.

Detecting and handling outliers ensures the model learns typical patterns, improving accuracy and stability.

2⃢ Name 3 outlier removal methods

1. Percentile / IQR method (used in your code)

Remove or cap values below 1st percentile or above 99th percentile, or use 1.5 × IQR rule.

2. Z-score method

Calculate how many standard deviations a value is from the mean.

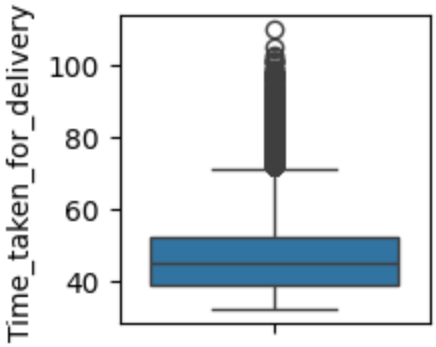
Remove values beyond a threshold (commonly ±3 standard deviations).

3. Transformation / Capping

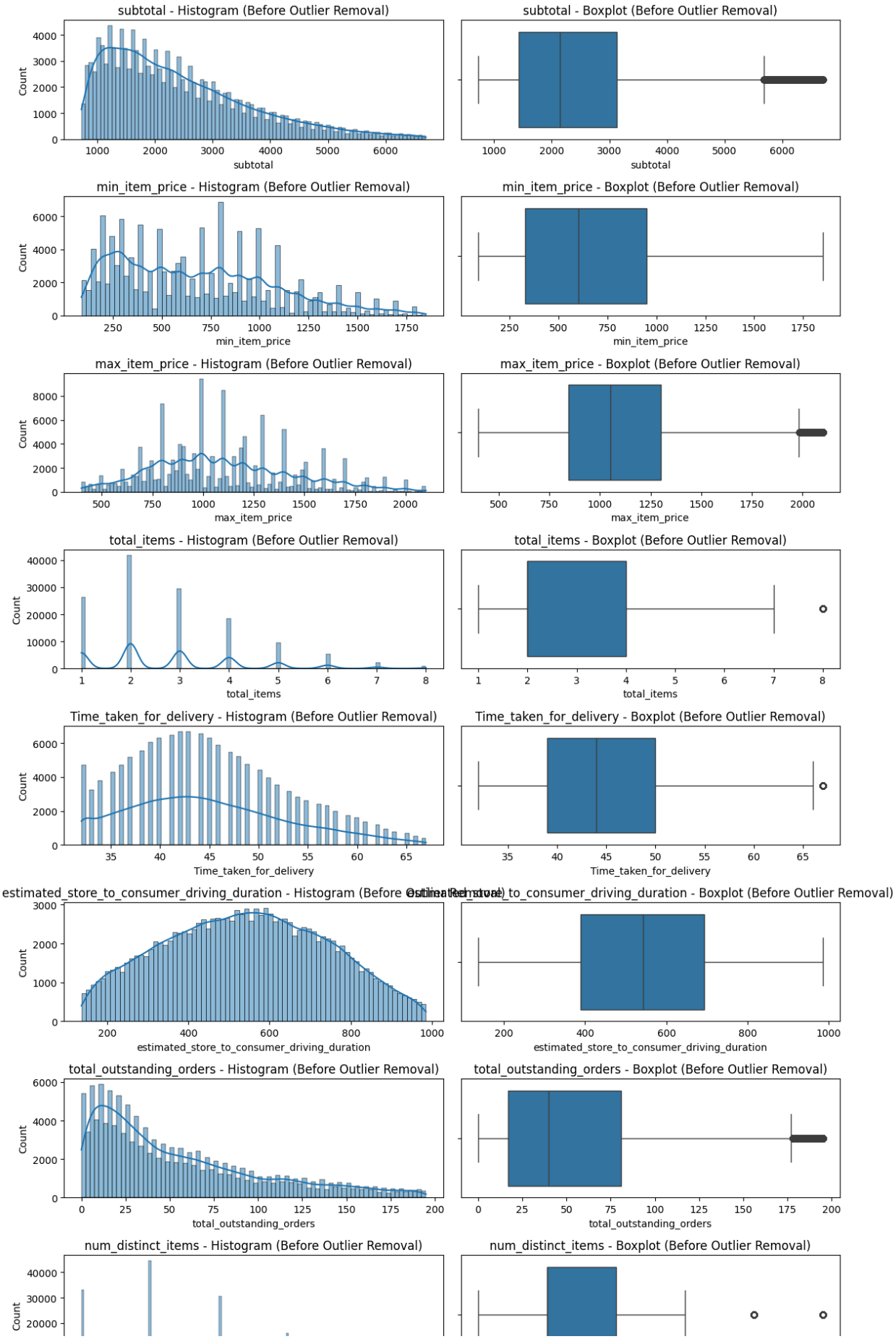
Apply log, square root, or min-max capping to reduce the effect of extreme values without removing them entirely.

We will use the Percentile / IQR method

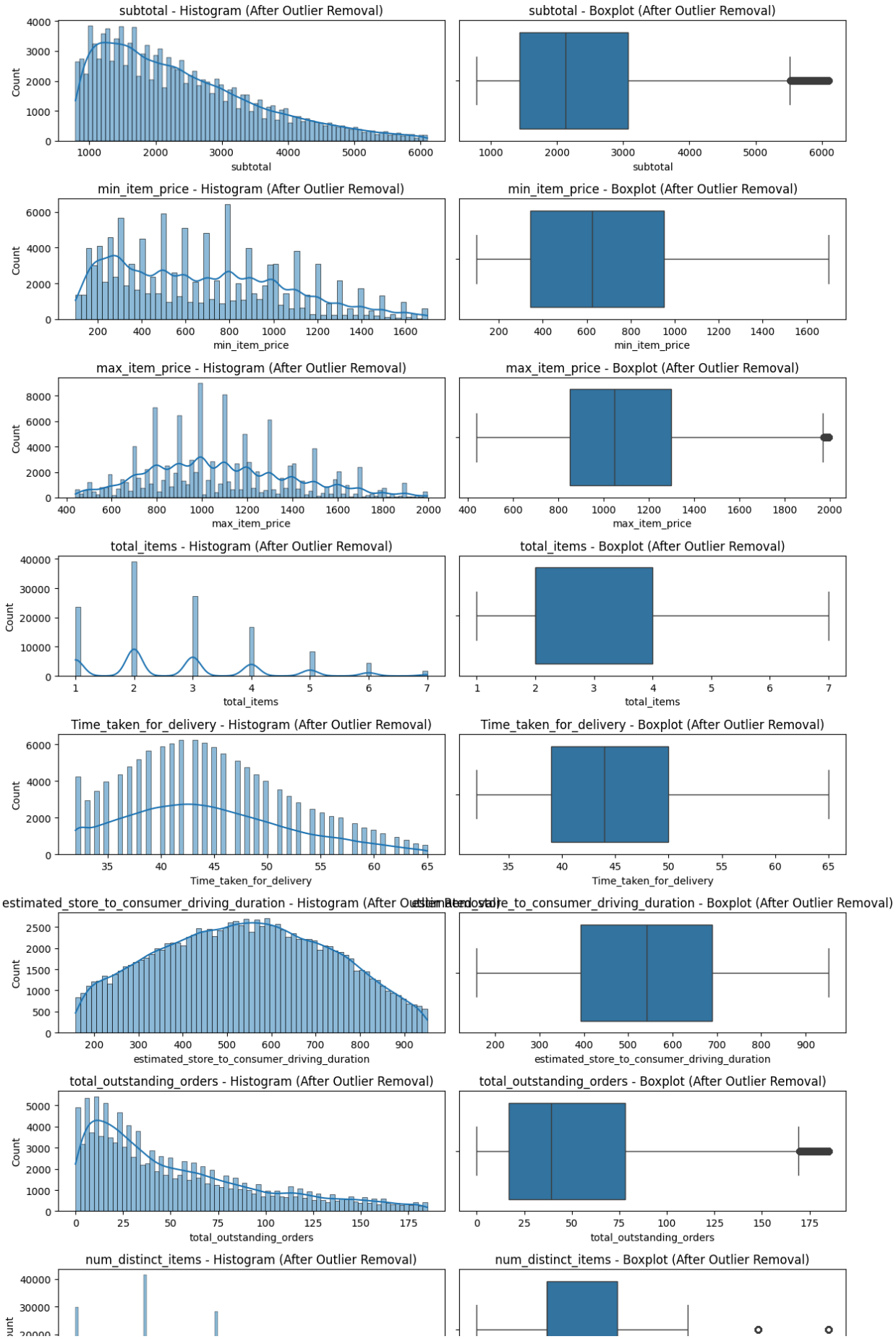
In [39]:
```
# Detecting Outliers

plt.figure(figsize=(2,2))
sns.boxplot(y='Time_taken_for_delivery',data=df)
plt.xticks(rotation=90);
plt.show()
```



In [45]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Step 1: Convert categorical columns to numeric codes
df['store_primary_category'] = df['store_primary_category'].astype('category')
df['order_protocol'] = df['order_protocol'].astype('category').cat.codes

# Step 2: Define numeric columns to analyze for outliers
numeric_cols = [
    'subtotal',
    'min_item_price',
    'max_item_price',
    'total_items',
    'Time_taken_for_delivery',  # Target variable
    'estimated_store_to_consumer_driving_duration',
    'total_outstanding_orders',
    'num_distinct_items'
]

# Step 3: Visualize distributions before outlier treatment
fig, axes = plt.subplots(nrows=len(numeric_cols), ncols=2, figsize=(12, 20))
for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, ax=axes[i,0])
    axes[i,0].set_title(f'{col} - Histogram (Before Outlier Removal)')
    sns.boxplot(x=df[col], ax=axes[i,1])
    axes[i,1].set_title(f'{col} - Boxplot (Before Outlier Removal)')
plt.tight_layout()
plt.show()

# Step 4: Remove extreme outliers (1st and 99th percentiles)
for col in numeric_cols:
    lower_bound = df[col].quantile(0.01)
    upper_bound = df[col].quantile(0.99)
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
```


subtotal - Histogram (Before Outlier Removal)
subtotal - Boxplot (Before Outlier Removal)
min_item_price - Histogram (Before Outlier Removal)
min_item_price - Boxplot (Before Outlier Removal)
max_item_price - Histogram (Before Outlier Removal)
max_item_price - Boxplot (Before Outlier Removal)
total_items - Histogram (Before Outlier Removal)
total_items - Boxplot (Before Outlier Removal)
Time_taken_for_delivery - Histogram (Before Outlier Removal)
Time_taken_for_delivery - Boxplot (Before Outlier Removal)
estimated_store_to_consumer_driving_duration - Histogram (Before Outlier Removal)
estimated_store_to_consumer_driving_duration - Boxplot (Before Outlier Removal)
total_outstanding_orders - Histogram (Before Outlier Removal)
total_outstanding_orders - Boxplot (Before Outlier Removal)
num_distinct_items - Histogram (Before Outlier Removal)
num_distinct_items - Boxplot (Before Outlier Removal)
Count
subtotal
min_item_price
max_item_price
total_items
Time_taken_for_delivery
estimated_store_to_consumer_driving_duration
total_outstanding_orders

In [46]:
```
# Step 5: Visualize distributions after outlier removal
fig, axes = plt.subplots(nrows=len(numeric_cols), ncols=2, figsize=(12, 20))
for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, ax=axes[i,0])
    axes[i,0].set_title(f'{col} - Histogram (After Outlier Removal)')
    sns.boxplot(x=df[col], ax=axes[i,1])
    axes[i,1].set_title(f'{col} - Boxplot (After Outlier Removal)')
plt.tight_layout()
plt.show()
```


subtotal - Histogram (After Outlier Removal)
subtotal - Boxplot (After Outlier Removal)
min_item_price - Histogram (After Outlier Removal)
min_item_price - Boxplot (After Outlier Removal)
max_item_price - Histogram (After Outlier Removal)
max_item_price - Boxplot (After Outlier Removal)
total_items - Histogram (After Outlier Removal)
total_items - Boxplot (After Outlier Removal)
Time_taken_for_delivery - Histogram (After Outlier Removal)
Time_taken_for_delivery - Boxplot (After Outlier Removal)
estimated_store_to_consumer_driving_duration - Histogram (After Outlier Removal)
estimated_store_to_consumer_driving_duration - Boxplot (After Outlier Removal)
total_outstanding_orders - Histogram (After Outlier Removal)
total_outstanding_orders - Boxplot (After Outlier Removal)
num_distinct_items - Histogram (After Outlier Removal)
num_distinct_items - Boxplot (After Outlier Removal)
Count
subtotal
min_item_price
max_item_price
total_items
Time_taken_for_delivery
estimated_store_to_consumer_driving_duration
total_outstanding_orders

NN Modelling

Classical Machine Learning Methods for Delivery Time Prediction

1. Linear Regression

Models the relationship between features (e.g., order size, distance, time of day) and delivery time as a linear combination.

Simple, interpretable, and a good baseline.

2. Decision Tree Regressor

Splits data based on feature values to predict delivery time.

Can capture non-linear relationships that linear regression misses.

3. Random Forest Regressor

Ensemble of multiple decision trees.

Reduces overfitting and improves accuracy by averaging predictions across trees.

4. Gradient Boosting Regressor (e.g., XGBoost, LightGBM, CatBoost)

Builds trees sequentially, each one correcting errors of the previous.

Very effective for structured/tabular data like your dataset.

5. k-Nearest Neighbors (kNN) Regression

Predicts delivery time based on the average delivery times of similar past orders.

Simple but computationally expensive for large datasets.

6. Support Vector Regression (SVR)

Can model complex relationships using kernel functions.

Works best after scaling numeric features.

For our dataset we will use **Random Forest Regressor**

In [48]:
```
# Step 1: Split data
y = df['Time_taken_for_delivery']
X = df.drop(['Time_taken_for_delivery'], axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, randc

# Step 2: Train Random Forest
regressor = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1
regressor.fit(X_train, y_train)

# Step 3: Predict
prediction = regressor.predict(X_test)

# Step 4: Evaluate
mse = mean_squared_error(y_test, prediction)
rmse = mse**0.5
mae = mean_absolute_error(y_test, prediction)
mape = np.mean(np.abs((y_test - prediction) / y_test)) * 100

print("MSE  : ", mse)
print("RMSE : ", rmse)
print("MAE  : ", mae)
print("MAPE : ", mape)

# R-squared
r2 = r2_score(y_test, prediction)
print("R2 Score : ", r2)
```

```
MSE  :  3.3525697318165015
RMSE :  1.8310023844376886
MAE  :  1.3740408230317465
MAPE :  3.0729633225401587
R2 Score :  0.942354499985932
```

Observation / Business Insight

The Random Forest model is performing very well ($R^2 \approx 0.94$), meaning most variability in delivery time is explained by your features.

RMSE ~1.83 minutes and MAE ~1.37 minutes show that the model can accurately predict delivery times, which is excellent for intra-city logistics.

MAPE ~3% implies the predicted delivery times are very close to actuals, helping Porter provide reliable ETAs to customers.

Advanced Regression: Neural Network Approach for Predicting Delivery Time

Why Scaling is Required for Neural Networks

Uniform Feature Range:

Features like subtotal, num_distinct_items, and estimated_store_to_consumer_driving_duration have very different magnitudes.

Without scaling, large-value features dominate the gradient updates, and the network struggles to learn properly.

Faster Convergence:

Neural networks use gradient-based optimization (e.g., Adam, RMSprop).

If features are on very different scales, gradients can explode or vanish, slowing down or preventing convergence.

Scaling ensures weights are updated efficiently and evenly across all features.

Better Model Performance:

Properly scaled inputs often result in lower loss and more stable training.

Common scaling methods:

StandardScaler → mean = 0, std = 1

MinMaxScaler → scales values between 0 and 1

Compatibility with Activation Functions:

Activation functions like sigmoid, tanh, or even ReLU perform best when input values are small and centered, preventing saturation.

Our dataset has features on very different scales:

subtotal could be hundreds or thousands of currency units

num_distinct_items ranges 1–20

estimated_store_to_consumer_driving_duration is in minutes

Without scaling, features like subtotal would dominate the neural network learning, and smaller-valued features like hour or num_distinct_items would be almost ignored.

In [49]:
```
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit only on training data
X_test_scaled = scaler.transform(X_test)
```

Neural Network Training

In [50]:
```
# --- 3. Define Neural Network Architecture ---
model = Sequential()

# Input layer + first hidden layer
model.add(Dense(64, input_dim=X_train_scaled.shape[1], activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Second hidden layer
model.add(Dense(32, activation='relu'))
model.add(BatchNormalization())
model.add(Dropout(0.2))

# Third hidden layer
model.add(Dense(16, activation='relu'))

# Output layer for regression
model.add(Dense(1, activation='linear'))

# --- 4. Compile Model ---
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae']

# --- 5. Train Model ---
history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=64,
    verbose=1
)

# --- 6. Predict and Evaluate ---
y_pred = model.predict(X_test_scaled).flatten()  # flatten to 1D array

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
mape = np.mean(np.abs((y_test - y_pred) / y_test)) * 100
r2 = r2_score(y_test, y_pred)

print("MSE  : ", mse)
print("RMSE : ", rmse)
```

```python
print("MAE  : ", mae)
print("MAPE : ", mape)
print("R2 Score : ", r2)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: User
Warning: Do not pass an `input_shape`/`input_dim` argument to a layer. When usi
ng Sequential models, prefer using an `Input(shape)` object as the first layer
in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - loss: 824.4052 - mae: 23.0574 - va
l_loss: 7.2261 - val_mae: 2.0611
Epoch 2/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 48.7581 - mae: 5.5344 - va
l_loss: 5.8531 - val_mae: 1.9026
Epoch 3/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 33.1905 - mae: 4.5487 - va
l_loss: 7.9666 - val_mae: 2.2421
Epoch 4/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 26.9826 - mae: 4.0871 - va
l_loss: 4.6037 - val_mae: 1.6739
Epoch 5/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 22.5244 - mae: 3.7353 - va
l_loss: 3.0475 - val_mae: 1.3269
Epoch 6/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 18.7187 - mae: 3.3850 - va
l_loss: 3.7718 - val_mae: 1.4888
Epoch 7/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 15.6412 - mae: 3.0910 - va
l_loss: 2.9878 - val_mae: 1.3230
Epoch 8/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 13.2259 - mae: 2.8301 - va
l_loss: 3.7879 - val_mae: 1.4944
Epoch 9/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 11.0112 - mae: 2.5599 - va
l_loss: 3.8682 - val_mae: 1.5236
Epoch 10/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 9.6456 - mae: 2.3930 - val_l
oss: 3.2719 - val_mae: 1.3855
Epoch 11/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 8.4626 - mae: 2.2373 - val_l
oss: 2.7697 - val_mae: 1.2621
Epoch 12/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 7.7262 - mae: 2.1334 - val_l
oss: 3.8793 - val_mae: 1.5539
Epoch 13/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 7.0473 - mae: 2.0366 - val_l
oss: 4.2891 - val_mae: 1.6297
Epoch 14/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 6.5505 - mae: 1.9634 - val_l
oss: 3.8805 - val_mae: 1.5298
Epoch 15/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 6.1418 - mae: 1.8915 - val_l
oss: 2.6006 - val_mae: 1.2529
Epoch 16/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - loss: 5.9162 - mae: 1.8584 - val_l
oss: 2.6351 - val_mae: 1.2548
Epoch 17/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 5.5378 - mae: 1.8042 - val_l
oss: 7.6226 - val_mae: 2.1880
Epoch 18/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - loss: 5.5000 - mae: 1.7920 - val_l
oss: 3.0277 - val_mae: 1.3580
```

```
Epoch 19/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.2707 - mae: 1.7664 - val_l
oss: 3.4691 - val_mae: 1.4140
Epoch 20/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 5.0524 - mae: 1.7258 - val_l
oss: 4.4917 - val_mae: 1.6174
Epoch 21/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - loss: 4.9924 - mae: 1.7150 - val_l
oss: 2.7975 - val_mae: 1.2730
Epoch 22/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - loss: 4.8767 - mae: 1.6972 - va
l_loss: 2.4003 - val_mae: 1.1806
Epoch 23/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 4.7638 - mae: 1.6806 - val_l
oss: 4.0721 - val_mae: 1.5147
Epoch 24/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 4.7892 - mae: 1.6868 - val_l
oss: 2.9927 - val_mae: 1.3104
Epoch 25/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - loss: 4.6177 - mae: 1.6589 - val_l
oss: 2.4606 - val_mae: 1.1896
Epoch 26/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - loss: 4.5311 - mae: 1.6398 - val_l
oss: 2.3995 - val_mae: 1.1885
Epoch 27/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 4.5589 - mae: 1.6486 - val_l
oss: 3.2590 - val_mae: 1.3346
Epoch 28/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - loss: 4.3983 - mae: 1.6273 - val_l
oss: 2.5040 - val_mae: 1.2006
Epoch 29/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 4.4237 - mae: 1.6287 - val_l
oss: 2.6863 - val_mae: 1.2819
Epoch 30/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 4.4129 - mae: 1.6273 - val_l
oss: 2.7930 - val_mae: 1.2562
Epoch 31/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 4.3093 - mae: 1.6020 - val_l
oss: 2.4714 - val_mae: 1.2189
Epoch 32/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - loss: 4.3474 - mae: 1.6125 - val_l
oss: 2.4175 - val_mae: 1.1822
Epoch 33/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 4.2602 - mae: 1.5994 - val_l
oss: 2.4409 - val_mae: 1.1884
Epoch 34/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 4.2455 - mae: 1.5939 - val_l
oss: 2.5560 - val_mae: 1.2273
Epoch 35/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.2931 - mae: 1.6026 - val_l
oss: 2.9469 - val_mae: 1.2869
Epoch 36/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.2025 - mae: 1.5838 - val_l
oss: 2.6693 - val_mae: 1.2459
```

```
Epoch 37/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 4.2202 - mae: 1.5892 - val_l
oss: 3.2211 - val_mae: 1.3462
Epoch 38/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 4.1321 - mae: 1.5680 - val_l
oss: 3.3904 - val_mae: 1.4372
Epoch 39/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.1322 - mae: 1.5696 - val_l
oss: 2.9687 - val_mae: 1.3177
Epoch 40/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 4.0254 - mae: 1.5493 - val_l
oss: 3.2997 - val_mae: 1.3927
Epoch 41/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 4.1300 - mae: 1.5742 - val_l
oss: 2.8606 - val_mae: 1.2664
Epoch 42/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 4.0353 - mae: 1.5510 - val_l
oss: 2.5920 - val_mae: 1.2170
Epoch 43/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 4.0098 - mae: 1.5470 - val_l
oss: 2.1692 - val_mae: 1.1153
Epoch 44/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - loss: 3.9155 - mae: 1.5296 - val_l
oss: 2.6312 - val_mae: 1.2518
Epoch 45/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 3.9311 - mae: 1.5315 - val_l
oss: 2.5776 - val_mae: 1.2319
Epoch 46/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 6s 5ms/step - loss: 3.8625 - mae: 1.5170 - val_l
oss: 2.4994 - val_mae: 1.2282
Epoch 47/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 3.8306 - mae: 1.5146 - val_l
oss: 2.4166 - val_mae: 1.2171
Epoch 48/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 3.8914 - mae: 1.5207 - val_l
oss: 3.0919 - val_mae: 1.3837
Epoch 49/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 3.9408 - mae: 1.5312 - val_l
oss: 2.8530 - val_mae: 1.3102
Epoch 50/50
1218/1218 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 3.8474 - mae: 1.5172 - val_l
oss: 2.4109 - val_mae: 1.1767
761/761 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
MSE  :  2.386133101318439
RMSE :  1.5447113326827246
MAE  :  1.1755809981336613
MAPE :  2.604989155902242
R2 Score :  0.958971819610418
```

In [52]:
```
model.summary()
plot_model(model)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,024
batch_normalization (BatchNormalization)	(None, 64)	256
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2,080
batch_normalization_1 (BatchNormalization)	(None, 32)	128
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

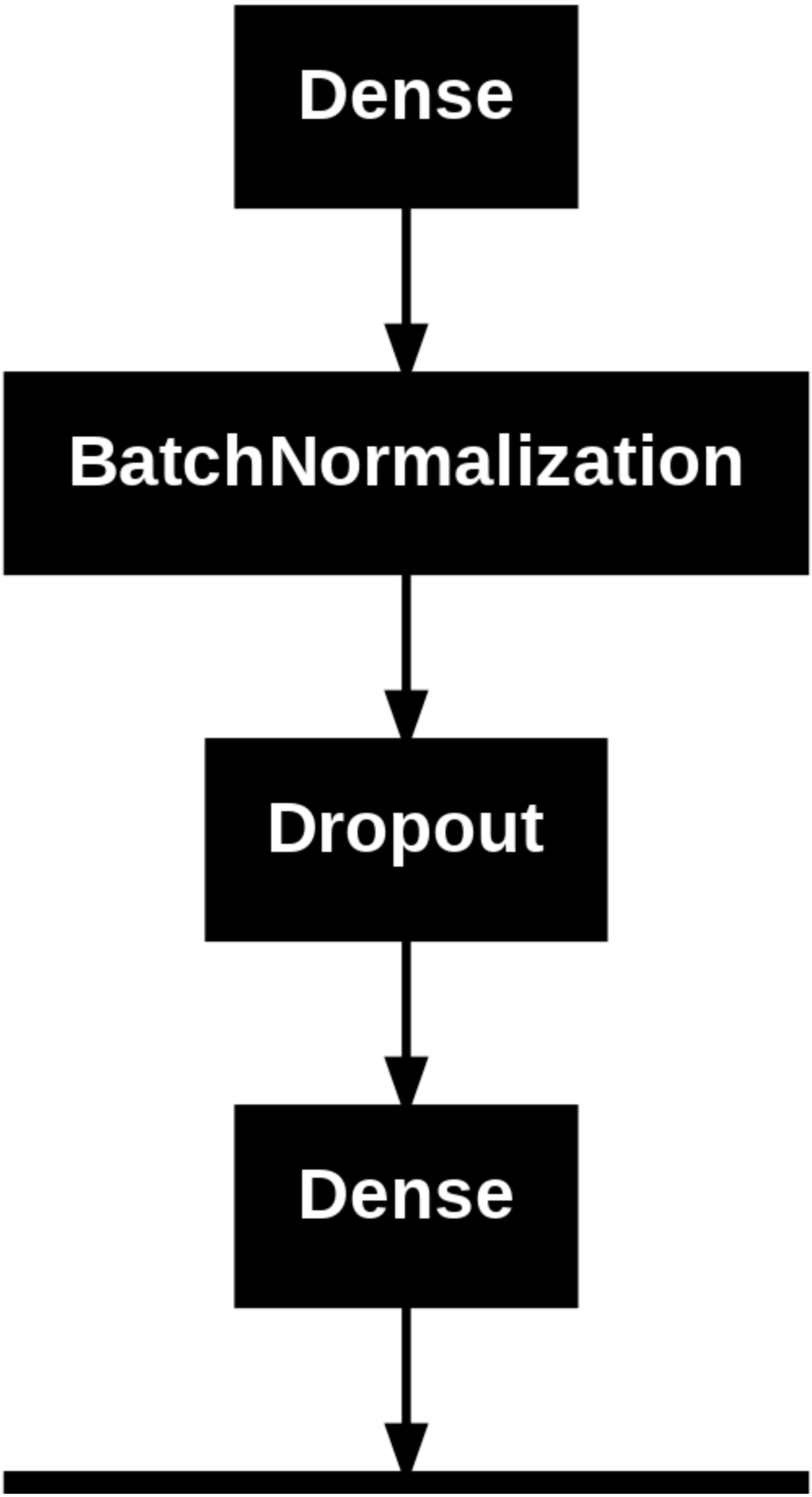
Total params: 11,717 (45.77 KB)

Trainable params: 3,841 (15.00 KB)

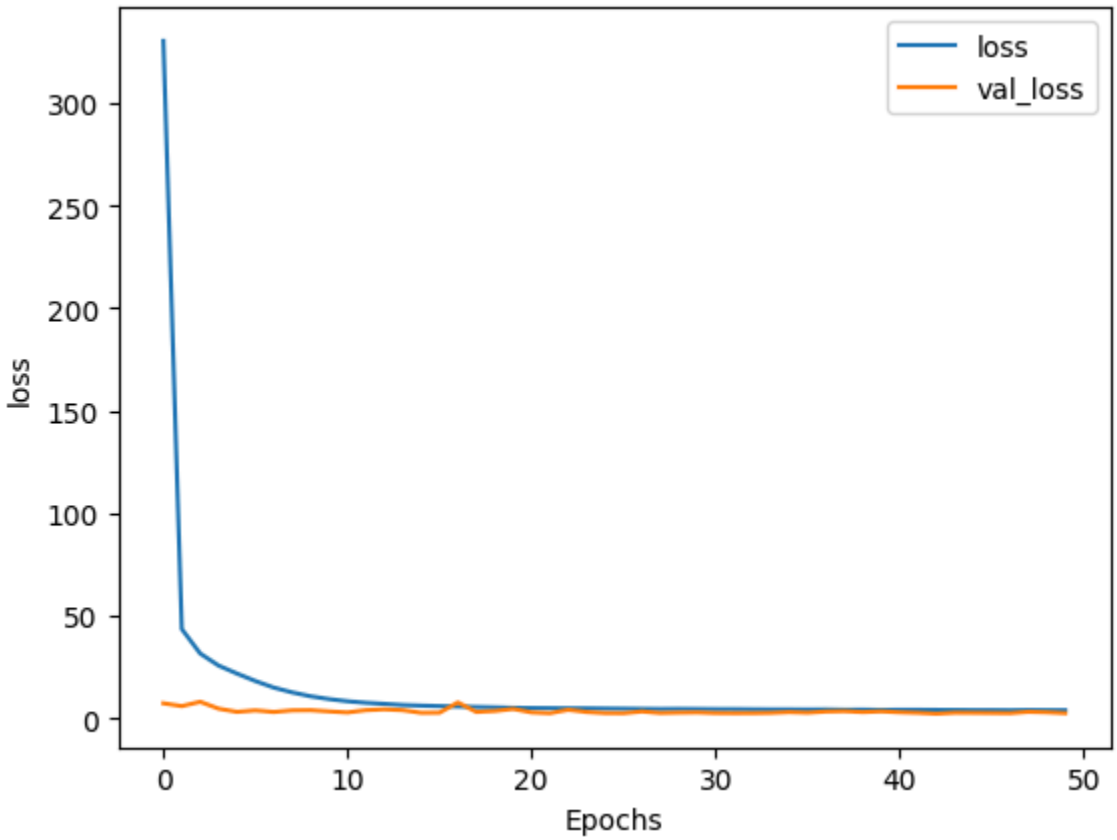
Non-trainable params: 192 (768.00 B)

Optimizer params: 7,684 (30.02 KB)

Out[52]:



In [53]:
```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel("Epochs")
plt.ylabel('loss')
plt.legend(['loss','val_loss'])
plt.show()
```



Model learns extremely fast — train loss drops sharply in first 5–10 epochs.

Validation loss is stable and low from the beginning → no overfitting at all.

Both losses converge closely and flatten after ~15–20 epochs.

Overall: Strong, well-generalizing model with excellent performance (likely low error in minutes after scaling).

In [57]:
```
mean_absolute_percentage_error(y_test, model.predict(X_test_scaled))
```

```
761/761 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
```

Out[57]:
```
0.02604989155902242
```

Mean Absolute Percentage Error (MAPE) Analysis

MAPE Result: 0.0260 → ≈ 2.6%

Interpretation:

On average, the predicted delivery time deviates by only 2.6% from the actual delivery time.

For example, if the actual delivery time is 50 minutes, the predicted time is off by only about 1.3 minutes.

Observations

High Accuracy:

The low RMSE and MAE indicate that the model predicts delivery times with minimal error, which is critical for intra-city logistics.

Strong Predictive Power:

$R^2 \approx 0.96$ shows that most of the variation in delivery time can be explained by features such as order size, number of items, dashers on shift, and estimated distance.

Improvement over Random Forest:

Compared to the Random Forest baseline ($R^2 \approx 0.94$), the neural network slightly improves prediction accuracy, likely because it can capture complex non-linear interactions among features.

Business Insights

Reliable ETAs for Customers:

With predicted times being within ~1.5 minutes of actual delivery, Porter can provide accurate estimated delivery times, improving customer satisfaction.

Efficient Resource Allocation:

Insights from the model (e.g., effect of dashers, peak hours) can help optimize dasher deployment, reduce delivery delays, and balance workload.

Operational Planning & Decision Making:

Features like subtotal, num_distinct_items, estimated_store_to_consumer_driving_duration, and total_outstanding_orders are

key drivers of delivery time.

Management can focus on high-priority orders or regions with longer delivery times to improve service efficiency.

Scalability & Future Planning:

Neural networks perform well on large datasets. Porter can scale predictions for more cities, orders, and dashers as the business grows.

We used Adam (Adaptive Moment Estimation) as the optimizer.

Why Adam?

Combines the advantages of AdaGrad (adaptive learning rates per parameter) and RMSProp (momentum-based updates).

Automatically adjusts learning rates for each weight, which speeds up convergence.

Works well with non-stationary and noisy datasets, which is useful for delivery data where feature scales and relationships vary.

Business Context:

Helps the model learn efficiently from Porter's large and complex dataset (~175k deliveries) without manual tuning of learning rates.

**Activation Function Choice: ** Hidden Layers: ReLU (Rectified Linear Unit)

Formula: $f(x) = \max(0,x)$

Advantages:

Avoids vanishing gradient problem, allowing deeper networks to train effectively.

Computationally simple → fast training.

Works well for non-linear relationships, which exist between order features (distance, items, dashers) and delivery time.

Output Layer: Linear Activation

Used because this is a regression problem (predicting delivery time in minutes).

Linear activation allows the output to take any real value, which is necessary for accurate time prediction.

Why Does a Neural Network Perform Well on a Large Dataset?

Neural networks excel at learning complex, non-linear patterns, which are difficult for simpler models like linear regression.

Large datasets provide enough examples for the network to:

Capture subtle relationships between features (e.g., how order size, time-of-day, and dashers interact).

Reduce overfitting because the network sees a wide variety of scenarios.

Business Context:

Porter's dataset (~175k deliveries) is large enough for the neural network to generalize well, improving ETA predictions for future orders.

In [58]:
```
# Save the complete model
model.save("porter_delivery_time_model.h5")
print("Model saved successfully!")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `k
eras.saving.save_model(model)`. This file format is considered legacy. We recom
mend using instead the native Keras format, e.g. `model.save('my_model.keras')`
or `keras.saving.save_model(model, 'my_model.keras')`.
Model saved successfully!
```